

Relatório de Laboratório

Laboratório 01 Relatório Final

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 Repositórios populares + Análise e Visualização
Grupo (trio)	João Victor Filardi Kayque Allan Tiago Boaventura Amaral
Link do repositório / GitHub Projects	https://github.com/JoaoFilardi/LabExperimentacao
Data de entrega	26/08/2026

1. Introdução

Plataformas como o GitHub concentram milhões de repositórios open-source, mas apenas uma fração pequena deles atinge grande popularidade, medida aqui pelo número de estrelas. Entender as características comuns a esses repositórios, como maturidade, volume de contribuição externa, cadência de releases, frequência de atualização, linguagem usada e taxa de resolução de issues, ajuda a separar o que é fato observável do que é apenas impressão sobre “boas práticas” de projetos open-source bem-sucedidos.

Este laboratório investiga seis Questões de Pesquisa (RQs) sobre os 1.000 repositórios mais populares do GitHub. Antes da coleta, o grupo formulou as seguintes hipóteses informais para cada uma:

RQ01 - Sistemas populares são maduros/antigos? Hipótese: sim. Ganhar popularidade leva tempo (adoção da comunidade, acúmulo de estrelas), então a maioria dos repositórios deve ter mais de 5 anos de criação.

RQ02 - Sistemas populares recebem muita contribuição externa? Hipótese: sim. Projetos populares atraem a atenção de muitos desenvolvedores e devem acumular um volume alto de Pull Requests aceitas.

RQ03 - Sistemas populares lançam releases com frequência? Hipótese: sim, mas não universal. Bibliotecas e frameworks populares tendem a versionar o que entregam; listas

awesome-*, materiais de estudo e roadmaps podem ter zero releases sem quebrar a tendência do conjunto.

RQ04 - Sistemas populares são atualizados com frequência? Hipótese: sim. A maior parte da amostra deve ter recebido push nos últimos 30 dias, embora alguns projetos famosos possam estar abandonados sem invalidar a hipótese para a maioria.

RQ05 - Sistemas populares são escritos nas linguagens mais populares? Hipótese: sim. Linguagens amplamente adotadas (Python, JavaScript/TypeScript, Java, Go) têm comunidades maiores e mais desenvolvedores capazes de contribuir.

RQ06 - Sistemas populares possuem alto percentual de issues fechadas? Hipótese: sim, mediana acima de 70–80%. Projetos populares tendem a ter mantenedores mais ativos, ainda que repositórios muito grandes acumulem issues abertas em número absoluto.

2. Contexto

Este laboratório é o primeiro experimento da disciplina e estabelece simultaneamente a base de mineração de dados e o processo de acompanhamento do grupo por GitHub Projects. Na Lab01S01 foi construída a consulta GraphQL própria e realizada a coleta inicial para 100 repositórios. Na Lab01S02, a consulta passou a percorrer os 1.000 repositórios e os dados foram persistidos em `repositorios_1000.csv`. Na Lab01S03, o CSV foi utilizado como fonte única para validação, análise estatística e geração dos gráficos.

O objeto de estudo é a amostra dos 1.000 repositórios mais estrelados no GitHub. Cada registro contém identificação, estrelas, data de criação, idade, PRs merged, releases, dias desde o último push, linguagem primária e dados de issues. A consulta utiliza diretamente a API GraphQL do GitHub, sem biblioteca de terceiros para acesso à API, conforme exigido pelo enunciado. Para RQ05, a referência externa adotada é o GitHub Octoverse 2025.

3. Metodologia

3.1 Principais Desafios

O principal desafio técnico foi realizar uma consulta GraphQL com campos agregados em uma amostra de 1.000 repositórios sem provocar falhas de gateway. O grupo observou problemas de 502 Bad Gateway e adotou lotes de 10 repositórios, usando `endCursor` e `hasNextPage` para paginação, totalizando 100 requisições para a amostra completa. O cliente também implementa `retry` para falhas transitórias, incluindo 429, 500, 502, 503 e 504, com espera exponencial.

Outro desafio foi trabalhar com distribuições assimétricas. RQ02, RQ03 e RQ04 apresentam caudas longas e outliers pelo critério IQR, tornando a mediana mais representativa do repositório típico do que a média. Também foi necessário distinguir zero de ausência de dado: zero releases significa ausência de GitHub Release; zero dias significa push no dia da coleta; e zero issues totais significa que não existe denominador para uma taxa convencional de fechamento.

3.2 Tomada de Decisões

- **Mediana em vez de média** como medida central em todas as RQs numéricas, por causa da assimetria à direita observada em quase todas as métricas, poucos repositórios “gigantes” distorcem a média (ex.: RQ02, média de 4.234 PRs vs. mediana de 768).
- **Outliers pelo método do IQR** (abaixo de $Q1 - 1,5 \times IQR$ ou acima de $Q3 + 1,5 \times IQR$), aplicado sistematicamente às RQs 01, 02, 03, 04 e 06. Outliers foram mantidos e discutidos, nunca removidos do CSV, por representarem repositórios reais (frameworks gigantes, projetos abandonados), não erro de linha.
- **Zero como valor legítimo, não dado ausente:** zero releases (RQ03) significa que o repositório não usa GitHub Releases; zero dias desde o último push (RQ04) significa push no mesmo dia da coleta; zero issues totais (RQ06) gera razão de 0,00% por definição (divisão por zero tratada no script), não “0% das issues foram fechadas”.
- **Linguagem N/A mantida como categoria própria** em vez de descartada, por representar repositórios legítimos sem linguagem primária detectada pela API (ex.: monorepos, listas awesome-*).
- **Fonte única para “linguagens mais populares”:** GitHub Octoverse, mantida como referência do início ao fim (RQ05 e RQ07), em vez de misturar índices diferentes.
- **Regra de não usar bibliotecas de terceiros** para a API do GitHub aplicada também ao script de snapshot do Projects, reaproveitando o mesmo `execute_query` de `graphql_client.py`.
- **Limite de WIP da coluna Doing:** detalhado na seção 3.3 (“Configuração do processo”), junto com a estrutura de colunas do board.

3.2 Etapas

O desenvolvimento do trabalho ocorreu de forma iterativa ao longo de três sprints sequenciais e uma etapa final de consolidação do relatório. A divisão de responsabilidades seguiu a atribuição real de Assignees nas Issues vinculadas ao GitHub Projects (v2).

Configuração do Processo

A gestão de tarefas do projeto foi estruturada e mantida no GitHub Projects (v2), vinculado diretamente ao repositório do grupo.

- **Colunas do Board:** O fluxo de trabalho foi mapeado em 5 colunas sequenciais: Backlog → To Do → Doing → Review → Done.
- **Política de WIP:** Máximo de 3 tarefas simultâneas na coluna Doing (1 por integrante).
- **Rastreabilidade Git ↔ Board:** Cada commit efetuado no repositório referenciou obrigatoriamente a tag numérica da Issue correspondente (ex.: #2, #5), permitindo que o GitHub vinculasse o histórico de código aos cartões do Projects para posterior auditoria docente.

Tabela de Execução por Sprint e Responsabilidade

Sprint	Entregas Efetuadas	Responsável(is)	Issues (nº)
Lab01S01	Setup do GitHub Projects (v2) com colunas, WIP e estrutura base do projeto em Python (.gitignore, .env.example, requirements.txt). Criada a query GraphQL inicial e validada a extração das RQs 01 e 02 em amostra de teste.	João Victor Filardi	#1, #2
Lab01S01	Implementação e validação individual da query GraphQL para extração de métricas da RQ 03 (Total de releases) e RQ 04 (Tempo desde última atualização) em amostra de teste.	Tiago Boaventura	#3
Lab01S01	Implementação e validação individual da query GraphQL para extração da RQ 05 (Linguagem primária) e RQ 06 (Razão de issues fechadas/totais).	Kayque Allan	#4

Sprint	Entregas Efetuadas	Responsável(is)	Issues (nº)
Lab01S02	Implementação do algoritmo de paginação GraphQL por cursores (endCursor, hasNextPage) para coleta contínua dos 1.000 repositórios e exportação para repositorios_1000.csv. Formulação das hipóteses informais para RQ01 e RQ02.	João Victor Filardi	#5, #6
Lab01S02	Validação da consistência dos dados minerados para RQ03 e RQ04 (análise de outliers e nulos) e redação das hipóteses informais correspondentes.	Tiago Boaventura	#7
Lab01S02	Validação de consistência dos dados para RQ05 e RQ06, redação das hipóteses informais e desenvolvimento do script de exportação automatizada de snapshot do board (snapshot_s02.csv).	Kayque Allan	#8

Sprint	Entregas Efetuadas	Responsável(is)	Issues (nº)
Lab01S03	Processamento estatístico (mediana, quartis) e geração dos gráficos de distribuição (boxplots e histogramas em Pandas/Seaborn) para RQ01 (Idade) e RQ02 (PRs Aceitas).	João Victor Filardi	#9
Lab01S03	Processamento estatístico e geração de gráficos de visualização para RQ03 (Releases) e RQ04 (Frequência de atualização).	Tiago Boaventura	#10
Lab01S03	Análise estatística da RQ05 (Popularidade de Linguagens), RQ06 (Issues Fechadas).	Kayque Allan	#11

Backlog 0/5 Estimate: 0 ... + This item hasn't been started	To do 0 Estimate: 0 ... + This item has to be done	Doing 0/3 Estimate: 0 ... + This is actively being worked on	Review 0/5 Estimate: 0 ... + This item is in review	Done 11 Estimate: 0 ... + This has been completed - LabExperimentacao #9 [Lab01S03] Análise Estatística e Gráficos — RQ01 e RQ02 - LabExperimentacao #11 [Lab01S03] Análise Estatística e Gráficos — RQ05, RQ06 e RQ07 - LabExperimentacao #10 [Lab01S03] Análise Estatística e Gráficos — RQ03 e RQ04 - LabExperimentacao #1 [Lab01S01] Setup do Kanban e Estrutura Inicial do Projeto - LabExperimentacao #2 [Lab01S01] Implementação GraphQL — RQ01 (Idade) e RQ02 (PRs Aceitas) - LabExperimentacao #3 [Lab01S01] Implementação GraphQL — RQ03 (Releases) e RQ04 (Última Atualização)
--	---	---	--	---

3.4 Ferramentas

Para a realização da coleta, processamento e gestão do projeto, a solução foi construída em Python utilizando estritamente a biblioteca padrão urllib.request para realizar a integração nativa via requisições HTTP à API GraphQL do GitHub, garantindo conformidade com a restrição de não utilizar bibliotecas de terceiros para mineração. Os dados extraídos foram persistidos em arquivos no formato CSV e, posteriormente, analisados por meio de scripts próprios desenvolvidos para validação, cálculo estatístico e geração de visualizações gráficas com Matplotlib.

No âmbito do processo e da governança ágil, utilizou-se o GitHub Projects (v2) para o rastreamento das Issues, controle da política de WIP e acompanhamento do fluxo de trabalho do grupo. Para contornar a limitação da API em relação ao histórico de transição de colunas do quadro, desenvolveu-se o script snapshot_projects.py, responsável pela extração periódica e acumulativa dos snapshots do board em CSV. Por fim, a referência adotada para a padronização e classificação das linguagens de programação mais populares ao longo de todo o estudo foi o relatório GitHub Octoverse 2025.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	data/hora da coleta – data de criação do repositório	Dias	GraphQL / CSV
RQ02	PRs merged	totalCount de pullRequests do repositório	Quantidade	GraphQL / CSV
RQ03	Total de releases	totalCount de releases do repositório	Quantidade	GraphQL / CSV
RQ04	Dias desde último push	data/hora da coleta – pushedAt	Dias	GraphQL / CSV
RQ05	Linguagem primária	primaryLanguage.name; N/A quando não detectada	Categoria	GraphQL / Octoverse
RQ06	Razão de issues fechadas	$(\text{issues_fechadas} / \text{issues_total}) \times 100$; 0% quando total = 0	Percentual	GraphQL / CSV

4. Resultados

4.1 Coleta de Dados

O arquivo repositorios_1000.csv contém exatamente 1.000 repositórios, sem nomes repetidos, e 13 colunas. As métricas numéricas utilizadas nas RQs 01, 02, 03, 04 e 06 possuem 1.000 registros válidos. Para RQ 05, 913 repositórios possuem linguagem primária detectada e 87 (8,70%) aparecem como N/A. Não foram identificados valores nulos nas métricas numéricas. Os outliers foram mantidos nas análises.

Estatística Descritiva - RQs 01 e 02

Métrica	RQ01 (Idade em Dias)	RQ02 (PRs Merged)
Total de Registros	1000	1000
Valores Nulos/Ausentes	0	0
Mínimo	5 dias	0
Máximo	6.703 dias (~18.3 anos)	103.316
Média	2.798,97 dias (~7.67 anos)	4.234,17
Primeiro Quartil (Q1 - 25%)	1.282,00 dias (~3.51 anos)	175
Mediana / Q2 (50%)	2.829,00 dias (~7.75 anos)	768
Terceiro Quartil (Q3 - 75%)	4.148,50 dias (~11.37 anos)	3.425
IQR (Q3 - Q1)	2.866,50 dias	3.250
Limites de Outliers	[-3017.75, 8448.25]	[-4700.00, 8300.00]
Quantidade de Outliers	0 (0.00%)	124 (12.40%)

Estatística Descritiva - RQs 03 e 04

Métrica	RQ03 (Total de Releases)	RQ04 (Dias desde o último push)
Total de Registros	1000	1000
Valores Nulos/Ausentes	0	0
Mínimo	0	0
Máximo	1.000	2.450
Média	126,16	113,45
Primeiro Quartil (Q1 - 25%)	0	0
Mediana / Q2 (50%)	39	1
Terceiro Quartil (Q3 - 75%)	146,2	49,2
IQR (Q3 - Q1)	146,2	49,2
Limites de Outliers	[-219.38, 365.62]	[-73.88, 123.12]
Quantidade de Outliers	92 (9,20%)	193 (19,30%)

Estatística Descritiva - RQs 05 e 06

Métrica	Valor	Linguagem	Quantidade	% da amostra
Total de Registros	1000	Python	228	22,80%
Valores Nulos/Ausentes	0	TypeScript	174	17,40%
Valores fora do intervalo [0, 100]	0	JavaScript	111	11,10%
Mínimo	0,00	N/A	87	8,70%
Máximo	100,00	Go	76	7,60%
Média	76,79	Rust	57	5,70%
Primeiro Quartil (Q1 - 25%)	67,19	C++	41	4,10%
Mediana / Q2 (50%)	86,48	Java	41	4,10%
Terceiro Quartil (Q3 - 75%)	96,55	Jupyter Notebook	24	2,40%
IQR (Q3 - Q1)	29,36	C	21	2,10%
Limites de Outliers	[23,15 ; 140,59]			
Quantidade de Outliers	60 (6,00%)			
Repositórios com <code>issues_total = 0</code> (razão definida como 0%)	43 (4,30%)			
Repositórios com 100% das issues fechadas	28 (2,80%)			
Repositórios com 0% das issues fechadas	43 (4,30%)			

4.2 Visualização Gráfica

RQ01: Sistemas populares são maduros ou antigos?

A idade mediana dos 1.000 repositórios é de 2.829 dias (~7,75 anos), com média de 2.798,97 dias (~7,67 anos), valores próximos, sem distorção forte. O repositório mais novo tem 5 dias; o mais antigo, 6.703 dias (~18,3 anos, praticamente a idade do GitHub). Nenhum outlier foi identificado pelo método IQR.

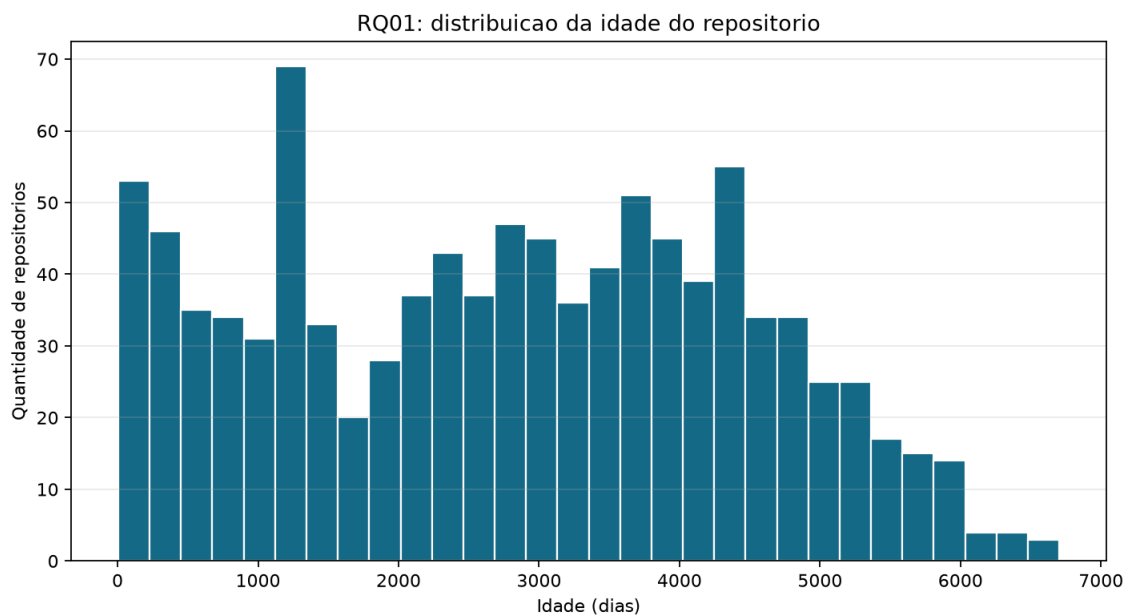


Figura 1. Histograma da idade dos repositórios.

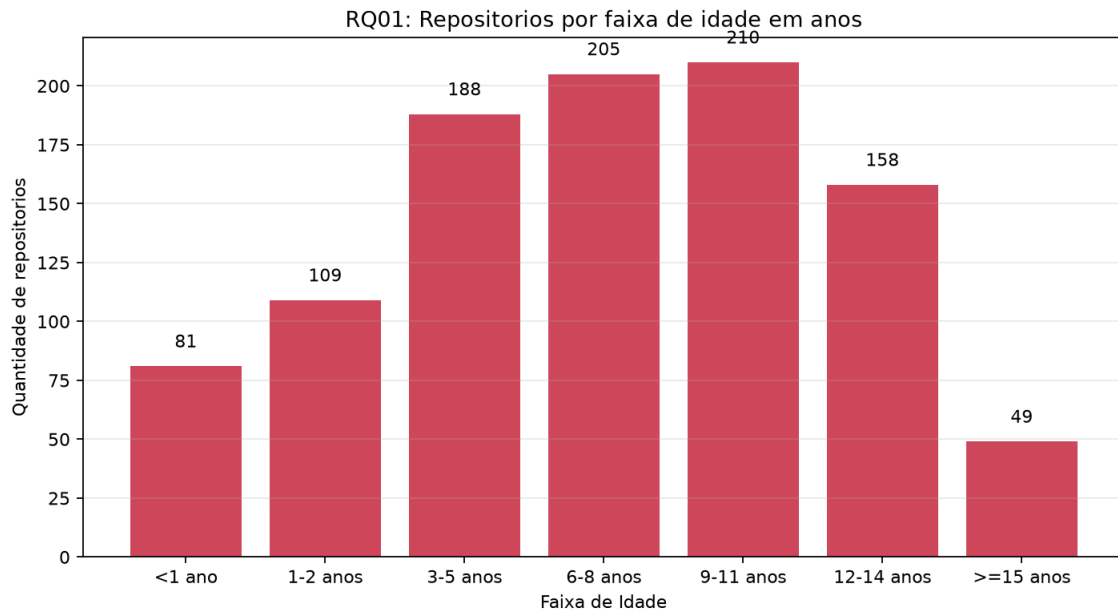
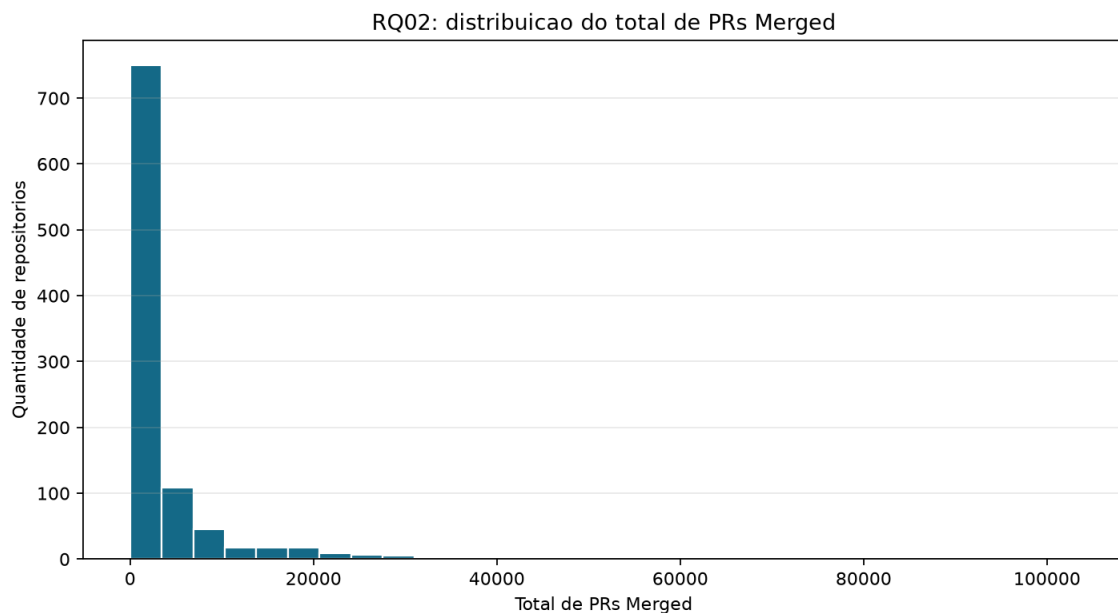


Figura 2. Distribuição dos repositórios por faixa de idade.

RQ02: Sistemas populares recebem muita contribuição externa?

A mediana de PRs merged é 768, bem abaixo da média (4.234,17), por causa de uma cauda de projetos hiperativos: o máximo chega a 103.316 PRs merged. Q1 = 175 e Q3 = 3.413,5 PRs. O método IQR identificou 124 outliers (12,4%), em geral acima de ~8.300 PRs merged, que correspondem a frameworks e projetos corporativos gigantes, não a erro de coleta.



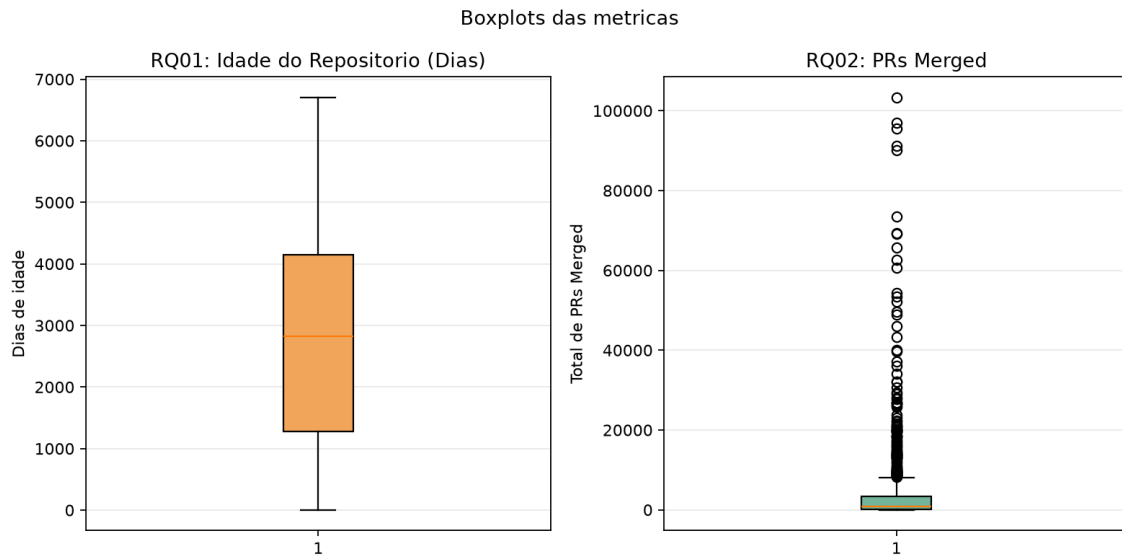


Figura 3. Boxplots das métricas de idade e PRs merged.

RQ03: Sistemas populares lançam releases com frequência?

A mediana é 39 releases; a média sobe para 126,16 por causa da cauda longa ($Q3 = 146,2$). 286 repositórios (28,6%) têm zero releases, não é dado ausente, é o caso de listas awesome-* e materiais que não usam o recurso GitHub Releases. O IQR aponta 92 outliers (9,2%), acima de ~366 releases; 21 desses repositórios (ex.: vercel/next.js, home-assistant/core) têm exatamente 1000 releases, sugerindo um teto da API.

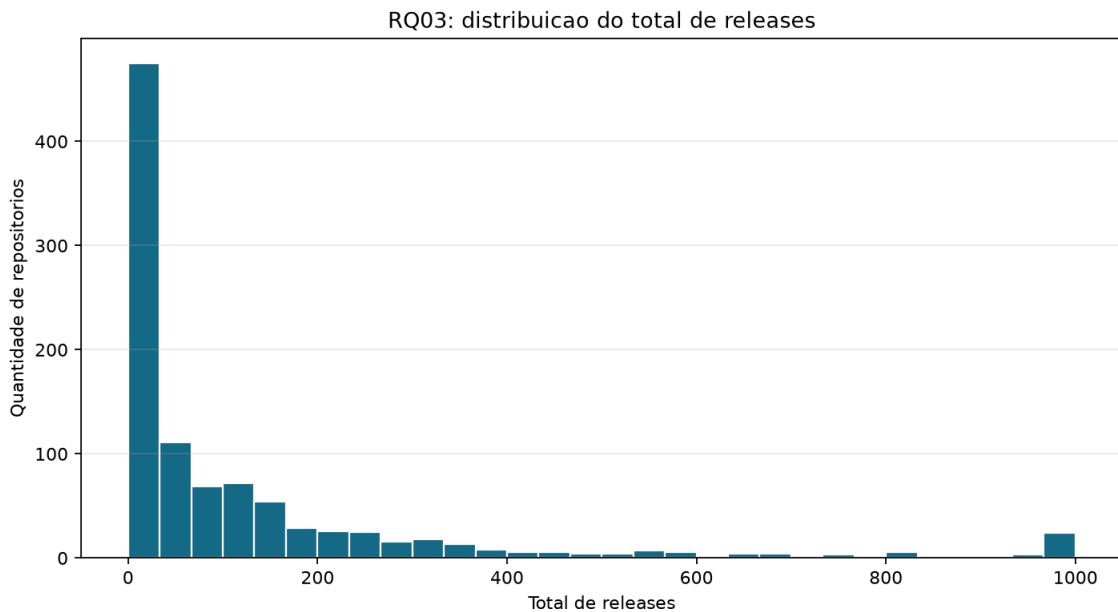


Figura 4. Histograma do total de releases.

RQ04: Sistemas populares são atualizados com frequência?

A mediana é de apenas 1 dia desde o último push, contra uma média de 113,45 dias, puxada por repositórios abandonados (máximo de 2.450 dias, ~6,7 anos, ex.: exacity/deeplearningbook-chinese). Q1 = 0 e Q3 = 49,2 dias. Agrupando por faixas: 430 repositórios (43,0%) tiveram push no mesmo dia da coleta, e 725 (72,5%) nos últimos 30 dias; no outro extremo, 67 (6,7%) não recebem push há mais de 2 anos. O IQR aponta 193 outliers (19,3%), em geral parados há mais de ~123 dias.

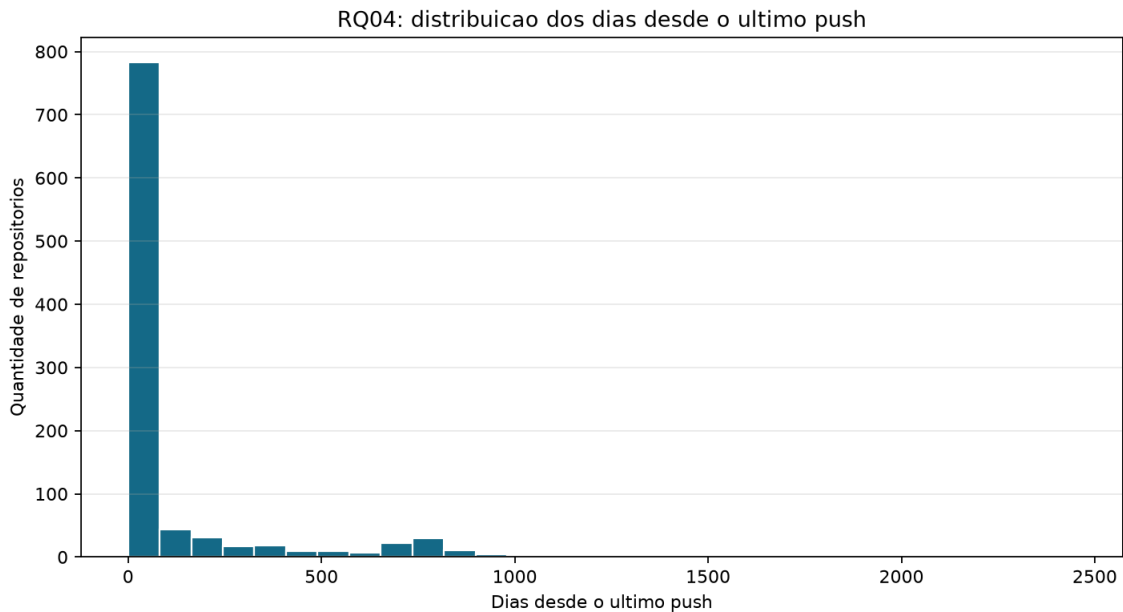


Figura 5. Histograma dos dias desde o último push.

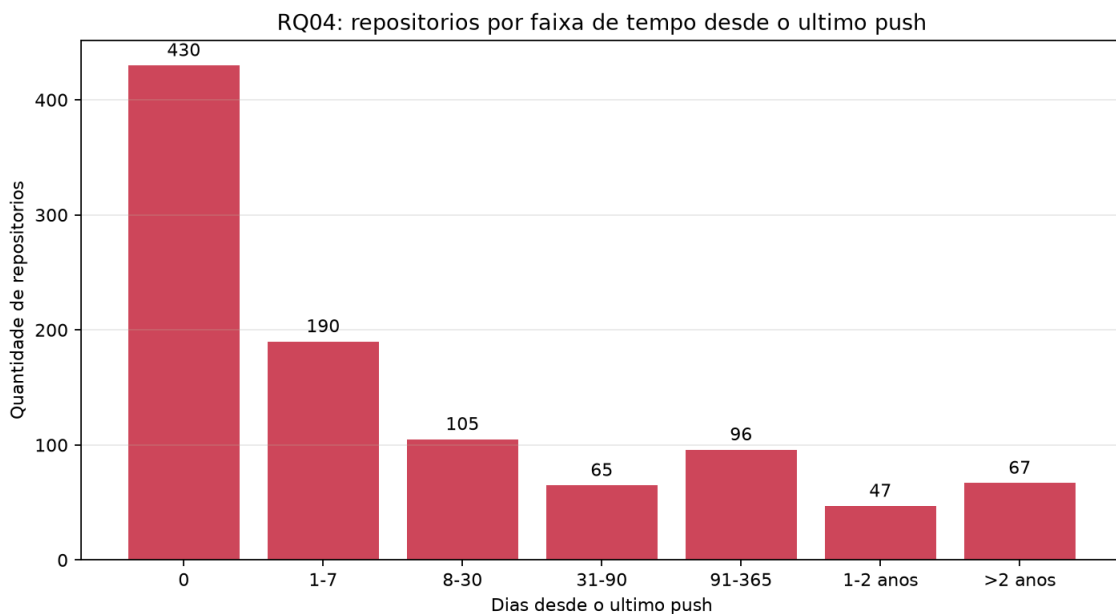


Figura 6. Distribuição por faixa de dias desde o último push.

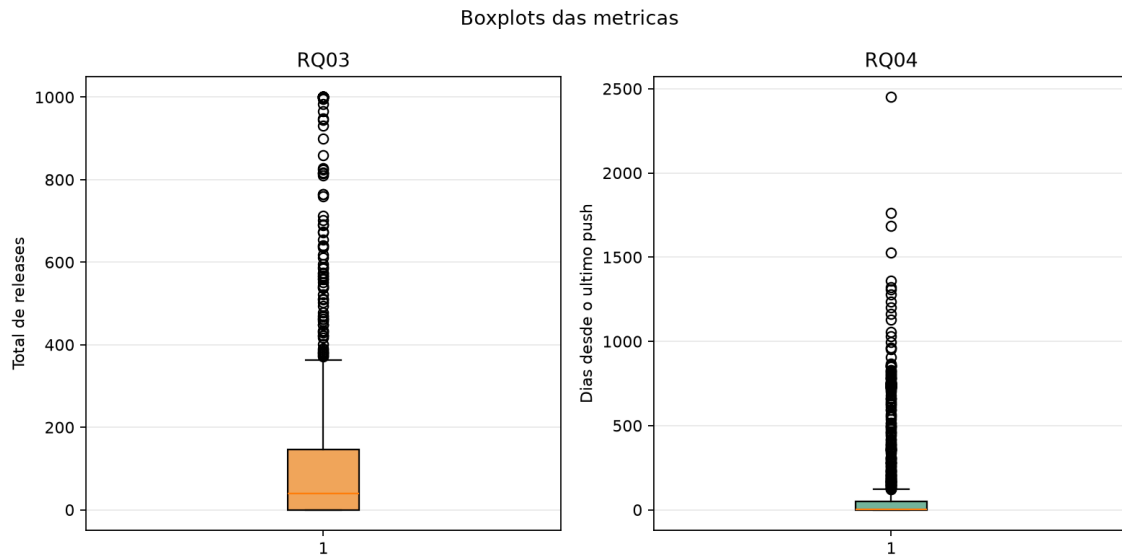


Figura 7. Boxplots das métricas de releases e dias desde o último push.

RQ05: Sistemas populares são escritos nas linguagens mais populares?

Usando o GitHub Octoverse como referência para “linguagens mais populares”, a amostra tem 43 linguagens distintas (fora N/A). As três mais frequentes, Python (228, 22,8%), TypeScript (174, 17,4%) e JavaScript (111, 11,1%), já somam 51,3% dos 1.000 repositórios; somando Go (76), Rust (57), C++ (41) e Java (41), sete linguagens concentram 72,8% da amostra. 87 repositórios (8,7%) não têm linguagem primária definida (N/A), tratados como categoria própria, não como erro de coleta.

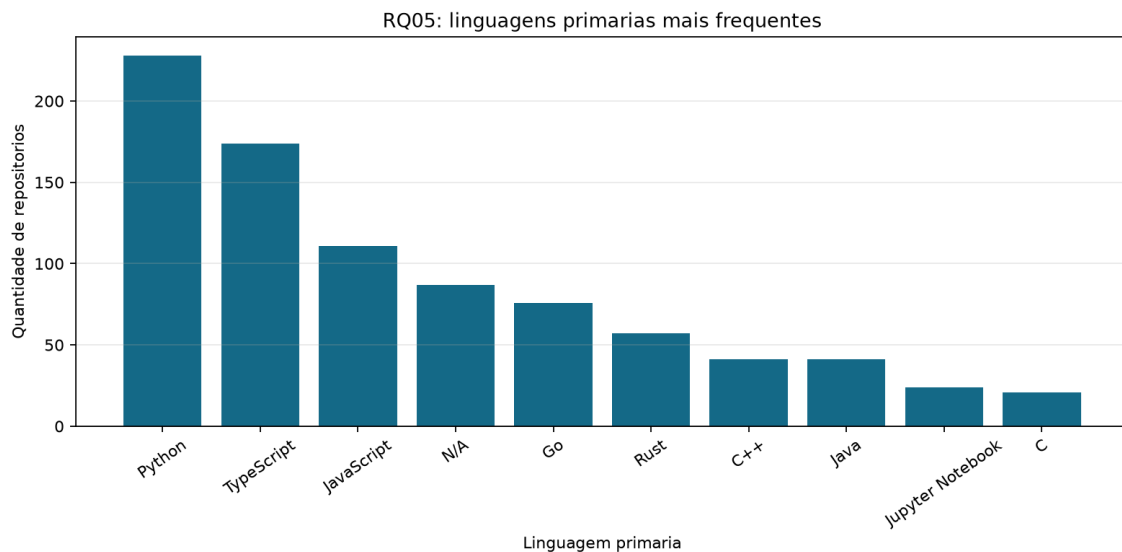


Figura 8. Frequência das linguagens primárias.

RQ06: Sistemas populares possuem alto percentual de issues fechadas?

A mediana de issues fechadas é 86,48%, acima da média de 76,79%, distribuição com assimetria à esquerda (poucos repositórios com percentual baixo puxam a média para baixo). Q1 = 67,19% e Q3 = 96,55%. Por faixas, 677 repositórios (67,7%) fecham entre 75% e 100% de suas issues, e apenas 65 (6,5%) ficam abaixo de 25%. 43 repositórios (4,3%) não têm nenhuma issue (razão definida como 0% por convenção do script) e 28 (2,8%) fecham exatamente 100%. O IQR aponta 60 outliers (6,0%), todos abaixo do limite inferior de 23,15%, não há outliers acima, já que 100% é o teto da métrica.

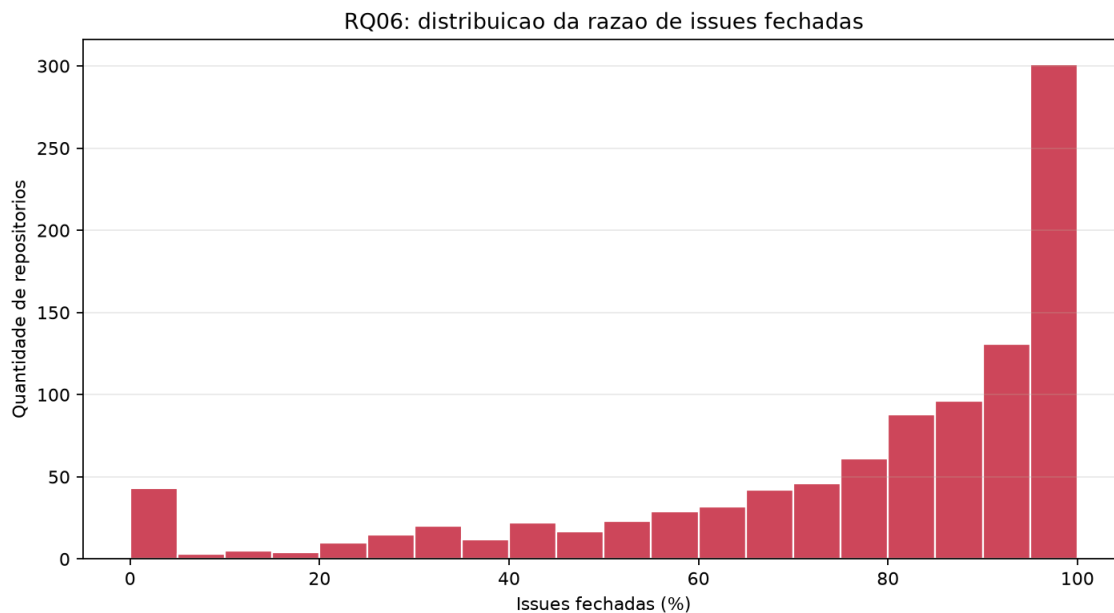


Figura 9. Histograma da razão de issues fechadas.

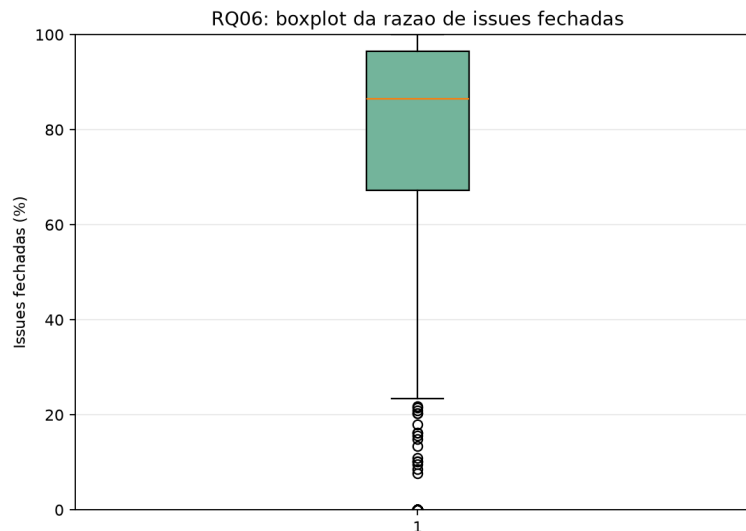


Figura 10. Boxplot da razão de issues fechadas.

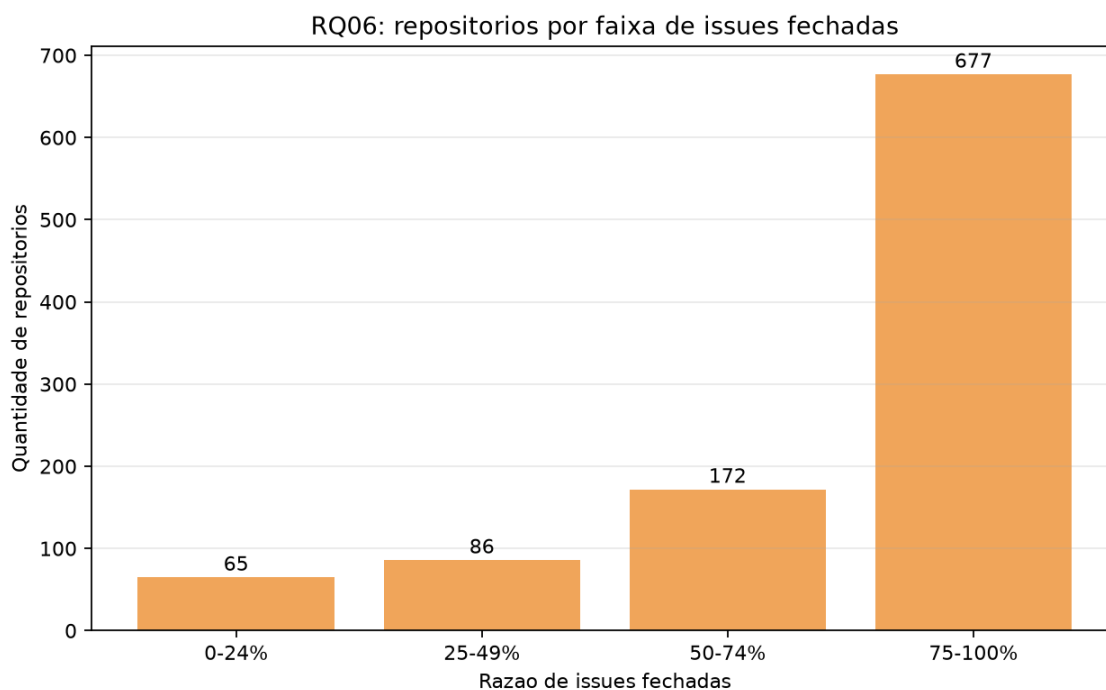


Figura 11. Distribuição por faixa de issues fechadas.

4.3 Discussão

Comparando a hipótese informal da Introdução com o resultado obtido, RQ a RQ:

RQ01 - Confirmada. Mediana de 7,75 anos e nenhum outlier confirmam a maturidade elevada, 72,3% da amostra tem 3 anos ou mais de criação.

RQ02 - Confirmada, com ressalva. O volume mediano (768 PRs) já é alto em termos absolutos, mas a média enganosa (4.234) mostra que uma minoria de projetos gigantes (12,4% de outliers) concentra a maior parte do volume total, a contribuição externa é real, mas desigual entre os repositórios.

RQ03 - Parcialmente confirmada. A mediana de 39 releases (> 0) confirma a tendência, mas 28,6% da amostra não usa o recurso Releases, mostrando que “lançar release” não é universal entre projetos populares, depende do tipo de projeto (biblioteca/framework vs. lista/material).

RQ04 - Confirmada para a maioria. 72,5% da amostra foi atualizada no último mês; a mediana de 1 dia é forte evidência de atividade. Ainda assim, 19,3% de outliers (repositórios parados há mais de ~4 meses) mostram que a popularidade não impede o abandono.

RQ05 - Confirmada. 72,8% da amostra concentra-se em sete linguagens do topo do GitHub Octoverse, com uma cauda longa de 43 linguagens distintas com poucos repositórios cada.

RQ06 - Confirmada. A mediana de 86,48% de issues fechadas é consistente com “alto percentual”; os 6% de outliers abaixo de 23% são, em geral, repositórios grandes e ativos que acumulam issues abertas, não indício de negligência generalizada.

5. Conclusão

O conjunto dos resultados indica um padrão coerente: os repositórios mais estrelados são, em geral, antigos, ativos e com forte participação da comunidade. A mediana de idade próxima de oito anos e a mediana de 768 PRs merged sustentam a ideia de projetos consolidados. Releases são comuns, embora uma parcela relevante não utilize GitHub Releases, enquanto a atualização recente é uma característica marcante da amostra.

As linguagens Python, TypeScript e JavaScript concentram mais da metade dos repositórios analisados, reforçando a presença de linguagens populares no conjunto de projetos mais estrelados. A razão de issues fechadas também é elevada.

As principais limitações são a seleção restrita aos 1.000 repositórios mais estrelados, o caráter pontual da coleta, o possível teto de 1.000 releases, a existência de repositórios sem linguagem primária e as distribuições assimétricas de algumas métricas. Com mais tempo, o grupo poderia repetir a coleta em diferentes datas, comparar a evolução das métricas e explorar relações entre estrelas e as demais variáveis.

5. Referências

- GITHUB. Octoverse 2025: A new developer joins GitHub every second as AI leads TypeScript to #1. GitHub Blog, 28 out. 2025.
- GITHUB. GraphQL API docs. Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 2026.
- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.